OWLSTORM.AI

Windows Compatibility Report

Anthropic Claude Code Course — uigen Project Setup

Prepared by: Owlstorm.ai | Date: March 17, 2026 | Platform Tested: Windows 11

Executive Summary

During completion of the Anthropic Claude Code course, Owlstorm.ai identified seventeen distinct compatibility and accuracy issues across the course setup instructions and the uigen project files. A user following the course materials exactly as written on Windows 11 would encounter blocking errors at every major setup stage with no guidance on how to resolve them.

The volume, nature, and distribution of these issues leads to one clear conclusion: the course materials were developed and tested exclusively on macOS or Linux and were never run on a Windows machine at any point prior to publication. This is not a matter of minor oversights — the errors span installation instructions, environment configuration, project scripts, and missing dependencies. Every major setup step fails on Windows.

Most tellingly, the project's package.json is missing zod entirely — a foundational library that the entire @ai-sdk ecosystem depends on. On macOS, zod is routinely present on a developer's machine from previous projects, meaning it works without being declared. On a clean Windows machine it simply does not exist. This is the classic hallmark of a project that has never been tested outside the developer's own environment.

This matters enormously for Anthropic's mission. Enterprise clients in manufacturing, fintech, and medical services — the sectors Anthropic most needs to win — run Windows. A developer course that completely fails on Windows does not just inconvenience users, it actively undermines confidence in Anthropic's developer tooling at the most critical first impression.

All seventeen issues have been identified, resolved or flagged, and documented in full below with root cause analysis, exact fixes applied, and recommended changes to the course materials.

> ⚠️ **Key Finding: This Course Was Never Tested on Windows**
>
> A Windows 11 user following this course exactly as written will encounter blocking errors at every major setup stage. The evidence strongly indicates the course developers had no familiarity with the Windows operating environment at the time of publication:
>
> - Node.js installation fails — Chocolatey conflict triggered by nodejs.org's own suggested Windows methods, no warning given
> - Claude Code install command is wrong — the CMD command provided does not work on Windows 11
> - Claude Code PATH not configured — installer places the binary but never registers it, not documented

- npm run dev fails completely — Unix-only NODE_OPTIONS syntax used throughout package.json, not resolvable even after applying the cross-env fix without a direct PowerShell workaround
- zod entirely missing from package.json — works on the developer's Mac by accident, fails on every clean Windows machine
- @ai-sdk/react missing from package.json — used throughout the codebase but never declared as a dependency
- zod version conflict — installing the missing @ai-sdk/react triggers a dependency resolution failure requiring an advanced npm flag no beginner would know
- NODE_OPTIONS persists in PowerShell session — the dev server workaround poisons subsequent npm install commands with a completely misleading error
- App code bug — input.trim() called on undefined value in MessageInput.tsx — crashes app on load on all platforms
- Infinite React render loop on first successful load — complete Windows system freeze requiring hard reboot to recover
- # memory shortcut does not automatically update CLAUDE.md — course teaches guaranteed behaviour that is actually model-dependent and can fail silently
- Ctrl+V clipboard image paste not supported on Windows 11 — core course workflow completely unavailable to Windows users with no documented alternative
- AI SDK v6 breaking changes — entire app non-functional due to 5 silent API changes between SDK versions, app never generates components on any platform

Root Cause Assessment:

The missing zod dependency is the most revealing indicator. Zod is not an obscure package — it is a core dependency of the entire Vercel AI SDK ecosystem used throughout this project. Its absence from package.json can only be explained by the project never having been installed on a clean machine. On macOS, zod accumulates silently across projects. On Windows, starting from zero, it simply is not there. A single test run of npm install on a clean Windows environment would have caught this immediately — along with the missing @ai-sdk/react, the broken NODE_OPTIONS scripts, and the incorrect install commands.

None of these seventeen issues include any guidance for Windows users anywhere in the course materials.

Issues Summary

Seventeen issues were identified across two categories: course instruction errors and project file errors.

#	Issue	Category	Severity	Status
1	Node.js installer triggers Chocolatey conflict	Instructions	Medium	✓ Resolved
2	Wrong Claude Code install command for Windows	Instructions	High	✓ Resolved
3	Claude Code PATH not configured after install	Instructions	Medium	✓ Resolved
4	No guidance to run PowerShell as Administrator	Instructions	Medium	✓ Resolved
5	.cjs file association — Windows opens dialog instead of running app	Instructions	Medium	✓ Resolved
6	NODE_OPTIONS syntax incompatible with Windows	Project Files	High	✓ Resolved
7	cross-env partial fix — PowerShell workaround required to start app	Project Files	High	✓ Resolved
8	Missing @ai-sdk/react dependency — app crashes on load	Project Files	High	✓ Resolved
9	.env file — no warning about quote characters breaking API key	Instructions	Medium	✓ Resolved
10	zod version conflict — @ai-sdk/react install fails without workaround flag	Project Files	High	✓ Resolved
11	zod not in package.json at all — cascading module not found errors	Project Files	High	✓ Resolved
12	NODE_OPTIONS persists in PowerShell session — breaks subsequent npm commands	Instructions	High	✓ Resolved
13	Code bug — input.trim() on undefined crashes app on load — all platforms	Project Files	High	✓ Resolved
14	Infinite React render loop — complete Windows system freeze, hard reboot required	Project Files	High	✓ Resolved
15	# memory shortcut does not auto-update CLAUDE.md — course inaccuracy	Instructions	Medium	✓ Identified
16	Ctrl+V clipboard image paste unsupported on Windows 11 — core workflow unavailable	Instructions	Medium	✓ Identified
17	AI SDK v6 breaking changes — 5 silent API changes make app completely non-functional	Project Files	High	✓ Resolved

Issue 1: Node.js Installer — Chocolatey Conflict via Suggested Windows Method	Severity: Medium	✓ Resolved

What Happened

The course directs Windows users to https://nodejs.org/en/download to install Node.js. The nodejs.org download page presents several suggested installation methods for Windows including winget and MSI installers. Following one of these suggested Windows methods triggered an attempt to install Chocolatey (a Windows package manager) as part of the 'Tools for Native Modules' step. On this system Chocolatey was already installed, causing a conflict that blocked the Node.js installation from completing entirely.

Importantly this was not a case of ignoring the recommended method — the Chocolatey conflict was triggered by following one of nodejs.org's own suggested Windows installation methods. The course provides no warning that this can happen or how to resolve it.

Root Cause

The nodejs.org download page's suggested Windows installation methods include a 'Tools for Native Modules' option that silently invokes Chocolatey. On machines where Chocolatey is already present — which is common on developer machines where it may have been installed previously by another tool or installer — this creates a conflict. The course assumes a completely clean Windows environment and gives no guidance on this failure mode.

Fix Applied

Bypassed the nodejs.org download page and all its suggested methods entirely. Used the direct native Windows MSI installer downloaded directly from the Node.js distribution server, and unchecked the 'Tools for Native Modules' option during setup to prevent Chocolatey from being invoked:

```
https://nodejs.org/dist/v22.22.1/node-v22.22.1-x64.msi
```

Recommended Fix

- Link directly to the native Windows MSI installer rather than the nodejs.org download page
- Add an explicit warning that the 'Tools for Native Modules' checkbox must be unchecked during installation to avoid the Chocolatey conflict
- Note that this conflict can occur even when following the suggested Windows methods on the nodejs.org page itself

---

Issue 2: Wrong Claude Code Install Command Provided for Windows	Severity: High	✓ Resolved

What Happened

The course provides the following Windows CMD command for installing Claude Code:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

This command did not work on Windows 11. The correct installation method for Windows is via PowerShell using the native installer:

```
irm https://claude.ai/install.ps1 | iex
```

Root Cause

The CMD-based command appears to be outdated or incorrect for current Windows 11 environments. The working PowerShell method uses the official Anthropic native installer script which handles the full installation automatically.

Fix Applied

Ran the PowerShell installer command above in an elevated PowerShell terminal (Run as Administrator).

Recommended Fix

- Replace the CMD install command with the PowerShell command in the course materials
- Add a note to open PowerShell as Administrator before running the install command
- Clarify that Windows Terminal should be used, not Command Prompt

---

Issue 3: Claude Code PATH Not Configured After Installation	Severity: Medium	✓ Resolved

What Happened

After running the installer, attempting to run claude in PowerShell produced:

```
claude : The term 'claude' is not recognized as the name of a cmdlet, function, script file, or operable program.
```

The installer placed claude.exe in the correct location but did not add that location to the Windows PATH environment variable.

Root Cause

The Claude Code installer correctly places claude.exe in C:\Users\[username]\.local\bin but does not automatically register this directory in the Windows user PATH. This is a known Windows-specific behaviour not covered anywhere in the course materials.

Fix Applied

Step 1 — Located claude.exe using PowerShell file search:

```
Get-ChildItem -Path $env:USERPROFILE -Recurse -Filter "claude.exe" -
ErrorAction SilentlyContinue
```

Step 2 — Permanently added the directory to the Windows user PATH:

```
[Environment]::SetEnvironmentVariable("PATH", $env:PATH +
";C:\Users\[username]\.local\bin", [EnvironmentVariableTarget]::User)
```

Step 3 — Closed and reopened PowerShell to pick up the updated PATH.

Recommended Fix

- Add a Windows-specific section to the Claude Code setup lesson covering the PATH configuration
- Include both commands above as standard steps for Windows users
- Explicitly note that PowerShell must be fully closed and reopened — not just a new tab — for PATH changes to take effect

---

Issue 4: No Guidance to Run PowerShell as Administrator	Severity: Medium	✓ Resolved

What Happened

Multiple steps throughout the setup require elevated (Administrator) PowerShell to function correctly on Windows 11. The course materials make no mention of this requirement at any point, which causes silent failures or permission errors depending on the system configuration.

Root Cause

On macOS and Linux, most terminal operations run with sufficient permissions by default. Windows 11 has User Account Control (UAC) which restricts what standard terminals can do. Running PowerShell as Administrator is a Windows-specific requirement with no equivalent in Unix-based systems.

Fix Applied

Right-clicked the Windows Terminal / PowerShell shortcut and selected 'Run as Administrator' before executing installation commands.

Recommended Fix

- Add a Windows prerequisite step at the start of the course: 'Open PowerShell by right-clicking and selecting Run as Administrator'

- Include a screenshot showing where the Run as Administrator option appears in Windows

---

Issue 5: .cjs File Association — Windows Opens File Dialog	Severity: Medium	✓ Resolved

What Happened

When attempting to run the app, Windows repeatedly displayed an 'Open with...' dialog asking which program to use to open node-compat.cjs. This happened because Windows did not recognise the .cjs file extension as a Node.js file type. The dialog appeared multiple times across different troubleshooting attempts, blocking progress each time.

Root Cause

The .cjs extension (CommonJS module format) is not registered as a known file type in a default Windows 11 installation. When the NODE_OPTIONS flag attempts to --require the file, Windows intercepts it as a file-open request rather than passing it as a runtime argument to Node.js.

This is a Windows-specific behaviour with no equivalent on macOS or Linux where file associations are handled differently.

Fix Applied

Dismissed the dialog and resolved the underlying NODE_OPTIONS syntax issue (see Issue 6) which eliminated the root cause of Windows intercepting the .cjs file reference.

Recommended Fix

- Add a note in the course materials warning Windows users that .cjs files may trigger an 'Open with' dialog
- Explain that the dialog should be dismissed and the underlying cross-env fix (Issue 6) applied instead
- Alternatively, register .cjs as a jsfile type in Windows as a setup prerequisite step

---

Issue 6 & 7: NODE_OPTIONS Syntax Incompatible with Windows + cross-env Missing	Severity: High	✓ Resolved

What Happened

Running npm run dev produced the following blocking error:

```
'NODE_OPTIONS' is not recognized as an internal or external command, operable program or batch file.
```

The app would not start at all. This error affects not just the dev script but four scripts in package.json.

Root Cause

The package.json scripts use Unix/macOS syntax for setting inline environment variables:

```
"dev": "NODE_OPTIONS='--require ./node-compat.cjs' next dev --turbopack"
```

Windows CMD and PowerShell do not support this syntax. The cross-env package exists specifically to solve this cross-platform incompatibility — it acts as a translator that rewrites the command into the correct syntax for whatever OS is running. However cross-env was not included in the project dependencies and was not mentioned anywhere in the course materials.

Fix Attempted — cross-env (Partial)

Step 1 — Installed cross-env as a dev dependency:

```
npm install --save-dev cross-env
```

Step 2 — Updated all four affected scripts in package.json to prefix with cross-env:

```
"dev": "cross-env NODE_OPTIONS='--require ./node-compat.cjs' next dev --turbopack"
"dev:daemon": "cross-env NODE_OPTIONS='--require ./node-compat.cjs' next dev --turbopack > logs.txt 2>&1 & echo 'Server started'"
"build": "cross-env NODE_OPTIONS='--require ./node-compat.cjs' next build"
"start": "cross-env NODE_OPTIONS='--require ./node-compat.cjs' next start"
```

Despite adding cross-env and updating all four scripts, npm run dev still did not work on Windows 11. The .cjs file association issue (Issue 5) continued to interfere, with Windows attempting to open node-compat.cjs as a file rather than passing it as a Node.js runtime argument.

Actual Working Fix — PowerShell Environment Variable

The only method that successfully started the app was setting the NODE_OPTIONS environment variable directly in PowerShell before running the dev server, bypassing npm run dev entirely:

```
$env:NODE_OPTIONS='--require ./node-compat.cjs'; npx next dev --turbopack
```

This sets the variable using native PowerShell syntax for the current session only, then runs the Next.js dev server directly via npx rather than through the npm script. This workaround is not documented anywhere in the course materials.

Recommended Fix

- Add cross-env to devDependencies in package.json before publishing — this is still the correct long-term fix
- Update all four scripts (dev, dev:daemon, build, start) with the cross-env prefix

- Add a Windows-specific note in the course explaining the PowerShell workaround as a fallback if npm run dev still fails
- Investigate and resolve the interaction between cross-env and the .cjs file association on Windows 11

Issue 8: Missing @ai-sdk/react Dependency — App Crashes on Load	Severity: High	✓ Resolved

What Happened

After resolving the NODE_OPTIONS issue the app compiled but immediately crashed with a module not found error visible in both the terminal and the browser (500 error on all pages):

```
Module not found: Can't resolve '@ai-sdk/react'
× ./src/lib/contexts/chat-context.tsx:9:1
GET / 500 in 24076ms
```

The app was completely non-functional — every page returned a 500 error.

Root Cause

The file src/lib/contexts/chat-context.tsx imports directly from @ai-sdk/react however this package is not listed in package.json dependencies. This means it is never downloaded when npm install runs during setup. The related package @ai-sdk/anthropic is listed but @ai-sdk/react is a separate package that must be declared independently.

This is not a Windows-specific issue — it would affect all platforms. It appears to be an oversight where the package was used in code but not added to the dependency list before the course zip was published.

Fix Applied

```
npm install @ai-sdk/react
```

Recommended Fix

- Add @ai-sdk/react to the dependencies section of package.json before republishing the uigen.zip file
- This is a single-line addition that eliminates the error for all users on all platforms

---

Issue 9: .env File — No Warning About Quote Characters Breaking API Key	Severity: Medium	✓ Resolved

What Happened

The course instructs users to add their Anthropic API key to the .env file. When opening the .env file with Windows Notepad and pasting the key, the entry was saved with surrounding quote characters:

```
ANTHROPIC_API_KEY="sk-ant-api-key-here"
```

This would cause all API calls to fail silently as Anthropic's servers would receive the key value with literal quote characters included, making authentication fail.

Root Cause

The course gives no guidance on the correct format for .env file values. Windows users opening the file in Notepad may naturally add quotes around the value, particularly if they are not developers. Some .env parsers handle quotes correctly but others include them as part of the value — causing the API key to be sent as "sk-ant-..." rather than sk-ant-... resulting in authentication failures.

Fix Applied

Removed the quote characters so the entry read:

```
ANTHROPIC_API_KEY=sk-ant-api-key-here
```

Recommended Fix

- Add a clear example in the course showing the correct .env format without quotes
- Add an explicit warning: 'Do not add quote marks around your API key'
- Consider recommending VS Code rather than Notepad for editing .env files, as VS Code provides better visual clarity

Issue 10: zod Version Conflict — @ai-sdk/react Install Fails	Severity: High	✓ Resolved

What Happened

After discovering the missing @ai-sdk/react package (Issue 8), attempting to install it produced a dependency conflict error that blocked the install entirely:

```
npm error ERESOLVE could not resolve
npm error While resolving: ai@6.0.116
```

```
npm error Found: zod@3.25.67
npm error Could not resolve dependency:
npm error peer zod@"^3.25.76 || ^4.1.8" from ai@6.0.116
npm error Fix the upstream dependency conflict, or retry this command with --force or --legacy-peer-deps
```

A beginner user would have no idea what this error means or how to resolve it. The install appeared completely broken.

Root Cause

The project has zod@3.25.67 installed as a peer dependency. However the latest version of @ai-sdk/react pulls in ai@6.0.116 which requires zod@3.25.76 or higher (or zod v4). The version pinned in the project is just 9 patch versions too old, causing npm's strict peer dependency resolver to refuse the installation entirely.

This is a cascading consequence of Issue 8 — not only is @ai-sdk/react missing from package.json, but the project's dependency versions are also out of sync with what @ai-sdk/react requires. Both problems need to be fixed together.

Fix Applied

Used the --legacy-peer-deps flag to bypass the strict version check and install anyway:

```
npm install @ai-sdk/react --legacy-peer-deps
```

This flag tells npm to use its older, more permissive dependency resolution behaviour and proceed with the install despite the version mismatch. In practice the version difference (3.25.67 vs 3.25.76) is minor and the app functions correctly.

Recommended Fix

- Update zod to the latest compatible version in package.json before republishing
- Add @ai-sdk/react to package.json dependencies at a pinned version that is compatible with the existing zod version
- Run npm install locally on a clean environment to verify all dependencies resolve cleanly before publishing the course zip
- This issue would not exist if Issue 8 were fixed properly — adding @ai-sdk/react at the correct pinned version to package.json would allow npm to resolve all dependencies correctly from the start

Issue 11: zod Not Listed in package.json — Cascading Module Errors	Severity: High	✓ Resolved

What Happened

Even after installing @ai-sdk/react with --legacy-peer-deps (Issue 10), the app continued to fail on load with a large cascade of module not found errors all related to zod:

```
Module not found: Can't resolve 'zod'
Module not found: Can't resolve 'zod/v3'
Module not found: Can't resolve 'zod/v4'
```

These errors appeared across multiple files deep inside node_modules — ai, @ai-sdk/react, @ai-sdk/gateway, and @ai-sdk/provider-utils were all unable to find zod. The app produced a 500 error on every page and was completely non-functional.

Root Cause

Zod is a data validation library used extensively by the entire @ai-sdk ecosystem. It is a direct dependency of ai, @ai-sdk/react, @ai-sdk/anthropic, and their sub-packages. Critically, zod is not listed anywhere in the project's package.json — not in dependencies, not in devDependencies. It is entirely absent.

On macOS or Linux, zod was likely already present on the developer's machine from a previous project or global install, meaning npm found it without it needing to be in package.json. This is a classic 'works on my machine' problem — a dependency that exists on the developer's system but is never formally declared in the project, so it never gets installed on a clean machine.

Windows users starting from a clean install have no zod anywhere on their system. When npm install runs during setup it only installs what is listed in package.json — and since zod is not listed, it is never installed. Every package that needs it then fails at runtime with module not found errors.

The newer versions of these packages import from zod/v3 and zod/v4 as sub-path exports — features only available in zod v4+. The project's zod version (installed via --legacy-peer-deps as a side effect) was too old to have these sub-paths, causing the cascade of errors seen across all zod imports.

Why This Happens More on Windows

This type of missing dependency issue is more likely to surface on Windows than macOS for several reasons:

- Windows developers are more likely to be starting from a completely clean environment with no pre-existing global packages
- macOS developers often accumulate packages across projects via Homebrew, nvm, or global npm installs that satisfy implicit dependencies
- The Node.js ecosystem on macOS tends to have more overlap between projects, making accidental dependency satisfaction more common
- Enterprise Windows machines are often locked down and stripped of developer tools, making clean-slate installations the norm

Steps Taken to Resolve

Step 1 — Identified that zod was completely absent from package.json:

```
"zod": not found anywhere in dependencies or devDependencies
```

Step 2 — Deleted node_modules entirely to start clean:

```
Remove-Item -Recurse -Force node_modules
```

Step 3 — Added zod v4 to package.json dependencies:

```
"zod": "^4.0.0",
```

Step 4 — Reinstalled all dependencies cleanly:

```
npm install --legacy-peer-deps
```

Step 5 — Restarted the app using the PowerShell workaround:

```
$env:NODE_OPTIONS='--require ./node-compat.cjs'; npx next dev --turbopack
```

Recommended Fix

- Add zod v4 to package.json dependencies immediately — this is the single most impactful fix in this entire report
- Run npm install on a clean Windows machine before publishing any course project zip to verify all dependencies resolve without errors
- Audit all other implicit dependencies that may exist on the developer's machine but are not declared in package.json
- Consider adding a package-lock.json to the zip file so that all users get exactly the same dependency versions that were tested

Issue 12: NODE_OPTIONS Persists in PowerShell Session — Breaks npm install	Severity: High	✓ Resolved

What Happened

After using the PowerShell workaround to start the dev server, subsequent npm install commands in the same PowerShell session failed with a confusing and unrelated-looking error:

```
npm error Error: Cannot find module './node-compat.cjs'
npm error Require stack: - internal/preload
npm error code MODULE_NOT_FOUND
```

Additionally npm produced a cascade of EPERM (permission denied) errors attempting to clean up node_modules directories. The install appeared completely broken and the error message gave no indication that the cause was an environment variable set earlier in the session.

Root Cause

When the PowerShell workaround command is run to start the app:

```
$env:NODE_OPTIONS='--require ./node-compat.cjs'; npx next dev --turbopack
```

The $env:NODE_OPTIONS variable is set for the entire duration of the PowerShell session — not just for the next dev command. Every subsequent Node.js process launched in that same session, including npm install, also inherits this variable and attempts to load node-compat.cjs before executing. Since node-compat.cjs only exists in the context of the running app and is not available to npm, every npm command fails.

This is a Windows PowerShell session scoping behaviour. On macOS/Linux using the Unix inline syntax NODE_OPTIONS='...' command the variable is scoped only to that single command and does not persist. PowerShell's $env: syntax sets the variable for the entire session unless explicitly cleared — a fundamental difference that the course materials do not acknowledge anywhere.

Fix Applied

Cleared the NODE_OPTIONS variable before running npm install:

```
$env:NODE_OPTIONS=""
```

Then deleted node_modules and reinstalled cleanly:

```
Remove-Item -Recurse -Force node_modules

npm install --legacy-peer-deps
```

After installation, set NODE_OPTIONS again only when starting the dev server:

```
$env:NODE_OPTIONS='--require ./node-compat.cjs'; npx next dev --turbopack
```

Why This Is a Significant Issue

This issue is particularly dangerous because the error message — 'Cannot find module node-compat.cjs' — gives no indication that the cause is an environment variable set earlier in the session. A user would have no reason to connect the two events. They would likely assume the npm install itself was broken and spend significant time debugging the wrong thing. This happened in our session during the attempt to fix the missing zod dependency, adding to an already complex chain of errors.

Recommended Fix

- Add a prominent Windows-specific warning in the course: 'After starting the dev server with the PowerShell workaround, always clear NODE_OPTIONS before running any npm commands'
- Provide the exact command: $env:NODE_OPTIONS="" as a standard step
- Alternatively, resolve the underlying NODE_OPTIONS issue in package.json properly so the workaround is never needed
- This issue would not exist at all if the cross-env fix in package.json worked correctly — eliminating the need for the PowerShell workaround entirely

Issue 13: Code Bug — input.trim() Called on Undefined — App Crashes on Load	Severity: High	✓ Resolved

What Happened

After resolving all dependency issues the app compiled successfully but still crashed on load with a JavaScript runtime error:

```
TypeError: Cannot read properties of undefined (reading 'trim')
at MessageInput (src\components\chat\MessageInput.tsx:43:41)
GET / 500 in 17788ms
```

The app returned a 500 error on every page load. The error occurred inside the MessageInput component which renders the chat input box.

Root Cause

In src/components/chat/MessageInput.tsx line 43, the code calls .trim() directly on the input variable:

```
disabled={isLoading || !input.trim()}
```

When the component first renders, input is undefined — it has not yet been initialised with a value. Calling .trim() on undefined throws a TypeError and crashes the entire page. This is a straightforward JavaScript null safety bug. The code assumes input will always be a string but does not handle the initial undefined state.

This bug would affect all platforms, not just Windows. However it was only discovered after resolving all the Windows-specific errors documented in Issues 1 through 12. On a Mac where the earlier issues do not exist, this bug may have been masked by other factors or simply not triggered in the testing environment.

Fix Applied

Replaced both instances of input.trim() in MessageInput.tsx with the optional chaining operator input?.trim():

```
disabled={isLoading || !input?.trim()}
```

The ?. operator — called optional chaining — means 'only call .trim() if input is not undefined or null, otherwise return undefined safely.' This prevents the TypeError on initial render while preserving the intended behaviour once input has a value.

There were two instances to fix — one on line 43 and one on line 46 in the same file.

Recommended Fix

- Fix both instances of input.trim() in MessageInput.tsx to use input?.trim() before republishing
- Review the codebase for other instances of direct property access on potentially undefined values

- This is not a Windows-specific issue but would block all users on all platforms from loading the app — it should be treated as a critical bug fix

Issue 14: Infinite React Render Loop — Complete Windows System Freeze	Severity: High	✓ Resolved

What Happened

When the app was first successfully loaded in the browser after resolving the dependency issues, it triggered a complete Windows system freeze. The mouse cursor stopped responding entirely — not just in the browser but across all programs and the Windows desktop. The only way to recover was a hard reboot using the physical power button.

This was not a browser crash or tab freeze — it was a full operating system lockup. Chrome, PowerShell, File Explorer and all other running programs became completely unresponsive simultaneously.

Root Cause

The freeze was caused by an infinite React render loop in chat-context.tsx. The useEffect hook that tracks anonymous work had fileSystem listed as a dependency:

```
useEffect(() => {
  if (!projectId && messages.length > 0) {
    setHasAnonWork(messages, fileSystem.serialize());
  }
}, [messages, fileSystem, projectId]);
```

Every time the effect ran it called fileSystem.serialize() which created a new object reference. This new reference caused React to detect a change in the fileSystem dependency, which triggered the effect again, which created another new object reference, which triggered the effect again — an infinite loop with no exit condition.

On macOS this type of infinite loop typically causes the browser tab to become unresponsive and crash, or slows the system noticeably but remains recoverable. On Windows 11 with older hardware the loop consumed 100% of available CPU within seconds, starving every other process including the Windows mouse cursor handler, causing a total system lockup requiring a hard reboot.

Why Windows Is More Vulnerable

Several factors make Windows more susceptible to a total freeze from this type of bug:

- Older Windows hardware with less CPU headroom has no spare capacity once a runaway process takes over
- Windows process scheduling under extreme CPU load can deprioritise the mouse cursor handler, making the system appear completely frozen

- macOS tends to isolate browser tab processes more aggressively, preventing a single runaway tab from consuming system-level resources
- The combination of mismatched dependency versions installed via --legacy-peer-deps likely made the render loop more aggressive than it would have been with correctly resolved dependencies

Fix Applied

Removed fileSystem from the useEffect dependency array in chat-context.tsx:

```
}, [messages, projectId]);
```

The fileSystem object does not need to be a dependency because the effect only needs to re-run when messages change, not when the fileSystem reference changes. Removing it breaks the infinite loop entirely. After applying this fix the app loaded successfully with no freeze, confirmed via browser automation.

Recommended Fix

- Remove fileSystem from the useEffect dependency array in chat-context.tsx before republishing
- Review all other useEffect hooks in the codebase for similar dependency array issues where object references could trigger infinite loops
- This bug would affect all platforms but is most dangerous on Windows where it causes a complete system freeze rather than a recoverable browser crash
- A hard reboot was required to recover — no keyboard shortcut or Task Manager intervention was possible as the system was completely unresponsive

Issue 15: # Memory Shortcut Does Not Automatically Update CLAUDE.md	Severity: Medium	✓ Identified

What Happened

The course instructs users that prefixing a message with # in Claude Code will automatically add that instruction to memory and persist it to the CLAUDE.md file. For example:

```
# Always use TypeScript strict mode
```

Following this instruction in Claude Code did not result in any update to CLAUDE.md. The note appeared to be acknowledged in the conversation but was never written to the file. When Claude Code was asked directly about this behaviour it provided the following self-assessment:

```
What # actually does: It adds a note into the conversation context. It is not
a documented system command that automatically writes to CLAUDE.md.

Why I failed to act on it: My instructions don't explicitly define # as 'write
to CLAUDE.md.' I treated it as conversational context rather than a persistent
update instruction. That was a judgment error on my part.
```

Root Cause

The # prefix is not a hard system-level feature — it is a convention that relies entirely on the model recognising the intent and choosing to act on it. There is no enforcement mechanism that guarantees a # prefixed message will be written to CLAUDE.md. The behaviour is model-dependent and can fail silently with no error or warning to the user.

This is a meaningful distinction between a guaranteed feature and an aspirational convention. The course presents it as the former when it is actually the latter.

Why This Matters

Developers building workflows that rely on CLAUDE.md staying up to date — for example teams maintaining shared coding standards or project-specific rules — cannot depend on # as a reliable mechanism. If the model decides not to act on it, instructions are silently lost with no indication that anything went wrong.

This is particularly relevant for enterprise deployments in regulated industries where consistency and auditability of AI behaviour matter. If Anthropic is positioning Claude Code for manufacturing, fintech and medical services clients, the distinction between guaranteed and aspirational features needs to be clearly communicated.

Recommended Fix

- Update the course to clearly state that # is a convention the model should act on, not a guaranteed system command
- Add guidance on how to manually update CLAUDE.md when # does not work as expected
- Consider implementing # as a true system-level command with guaranteed write behaviour in a future Claude Code release
- In the meantime instruct users to verify CLAUDE.md has been updated after using # and manually edit it if not

Issue 16: Clipboard Image Paste (Ctrl+V) Not Supported on Windows 11	Severity: Medium	✓ Identified

What Happened

The Claude Code course demonstrates using Ctrl+V to paste a screenshot directly into the Claude Code terminal window to provide visual context for changes. This workflow does not work on Windows 11. Attempting to paste a screenshot taken with Win+Shift+S produces no result in Claude Code.

Root Cause

Windows 11 stores screenshots taken with Win+Shift+S as raw bitmap data in the clipboard — not as a file. Terminal applications including Claude Code on Windows cannot read raw bitmap clipboard data. macOS handles clipboard images differently, storing them in a format that terminal applications can access directly, which is why the workflow works seamlessly on Mac but fails entirely on Windows.

This is a known limitation with an open GitHub issue requesting native Windows support: https://github.com/anthropics/claude-code/issues/26679. The issue has not yet been resolved at time of writing.

Why This Matters for the Course

The course demonstrates Ctrl+V image paste as a core workflow for providing visual context to Claude Code — showing it a screenshot of the UI area you want to change. Windows users following along cannot use this workflow at all. The course gives no alternative instructions for Windows users, leaving them unable to replicate a key demonstrated feature.

Workarounds for Windows Users

- Use Snipping Tool → Save As to save the screenshot as a file first, then reference the file path in Claude Code
- Drag and drop an image file directly into the Claude Code terminal window
- Type the file path of a saved image directly in your message to Claude Code
- Use a tool like ShareX which auto-saves screenshots to disk automatically, making them immediately available as files

Recommended Fix

- Add a Windows-specific note in the course explaining that Ctrl+V image paste is not supported and providing the workarounds above
- Track and link to the open GitHub issue so Windows users know this is a known limitation being worked on
- Consider implementing native Windows clipboard image support as a priority given the enterprise Windows user base Anthropic is targeting

Issue 17: AI SDK v6 Breaking Changes — Code Written for v4/v5, App Completely Non-Functional	Severity: High	✓ Resolved

What Happened

Even after resolving all dependency and configuration issues, the app failed to generate components correctly. Messages appeared in the chat but no preview was ever rendered. Claude responded with text like 'I'll create a simple Card component for you' but nothing

appeared in the preview panel. This was ultimately traced to a fundamental version mismatch — the uigen codebase was written for Vercel AI SDK v4/v5 but the installed package was v6, which introduced five major breaking API changes that silently broke the entire AI integration. This required Claude Code CLI to diagnose and resolve.

Breaking Change 1 — Messages Not Rendering

The Message type was replaced by UIMessage in AI SDK v6. The old code used a fallback to message.content which always returned null in v6 because v6 messages only have .parts. Fixed by switching to UIMessage and removing the content fallback.

Breaking Change 2 — Chat Never Sent Data to Server

In AI SDK v6, useChat no longer accepts api, body, or initialMessages as direct options — they were silently ignored with no error or warning. Every chat submission was going nowhere. Fixed by explicitly creating a DefaultChatTransport with the correct body and passing it via the transport option.

Breaking Change 3 — Preview Never Updated

The onToolCall callback in AI SDK v6 only fires for client-side tools. Since the uigen tools run server-side, onToolCall was never called and the client virtual file system stayed permanently empty — meaning no component was ever rendered in the preview panel. Fixed by watching messages in a useEffect and applying completed tool operations to the client VFS.

Breaking Change 4 — Tool Display Broken

In AI SDK v6, tool parts have type 'tool-{toolName}' (e.g. 'tool-str_replace_editor') instead of 'tool-invocation', and the state is 'output-available' instead of 'result'. The rendering code was checking for old types and therefore never displayed tool activity.

Breaking Change 5 — API Route Response Format Changed

The API route used the old toDataStreamResponse() method which was replaced by toUIMessageStreamResponse() in v6. Tools were also updated from parameters: to inputSchema: with jsonSchema(), and hydration errors were fixed with explicit panel IDs and ssr: false.

Root Cause

The uigen project was built and tested against AI SDK v4/v5. When package.json was published with ai: ^6.0.116 (the ^ caret means 'latest v6'), the installed version introduced five breaking API changes the codebase was never updated to handle. Because JavaScript does not enforce API contracts at install time, none of these breaking changes produced errors during npm install — they failed silently at runtime, making them extremely difficult to diagnose without deep knowledge of what changed between SDK versions.

This is the root cause of the most confusing symptoms: messages appearing in the chat with no preview, Claude responding but nothing rendering, and tool calls appearing to work but producing no output. It also explains why the app appeared to work on the developer's machine — they were likely running an older installed version of the AI SDK that matched what the code expected.

Recommended Fix

- Update the entire uigen codebase to use AI SDK v6 APIs — all five breaking changes above must be addressed
- Pin the ai package to a specific version (e.g. '6.0.116' not '^6.0.116') to prevent future unexpected breaking changes from auto-installing
- Run npm test before publishing any course project zip to verify the test suite passes against the installed versions
- Add a CI/CD check that runs tests on dependency updates before republishing course materials

Recommended Changes for Anthropic Course Team

The following table summarises all recommended changes. Items marked as project file changes require updating the uigen.zip before republishing. Instruction changes require updates to the Skilljar course content.

#	Type	Action Required	Priority
1	Project File	Add zod v4 to package.json dependencies — most impactful single fix	*Immediate — cascading failures without it*
2	Project File	Add @ai-sdk/react to package.json dependencies at correct pinned version	*Immediate — blocks all users*
3	Project File	Add cross-env to devDependencies and prefix 4 scripts — document PowerShell workaround as fallback	*Immediate — npm run dev never works on Windows*
4	Project File	Fix zod version conflict — pin compatible versions across all @ai-sdk packages	*Immediate — install fails without workaround flag*
5	Course Content	Replace CMD install command with PowerShell command for Windows	*High — incorrect instruction*
6	Course Content	Add Windows PATH fix instructions for Claude Code post-install	*High — blocks Windows users*
7	Course Content	Add 'Run as Administrator' guidance for Windows PowerShell	*Medium — causes silent failures*
8	Course Content	Warn users to uncheck 'Tools for Native Modules' during Node.js install	*Medium — Chocolatey conflict*
9	Course Content	Add .env format example showing no quotes around API key	*Medium — breaks API auth*
10	Course Content	Add note about .cjs file association dialog on Windows	*Medium — confusing for users*

#	Type	Action Required	Priority
11	Process	Test all course materials on a clean Windows 11 machine before publishing	*Systemic — root cause of all issues*
12	Course Content	Warn users to clear $env:NODE_OPTIONS before running npm commands after starting dev server	*High — causes misleading unrelated errors*
13	Project File	Fix input?.trim() null safety bug in MessageInput.tsx — crashes app on all platforms	*Immediate — blocks all users on all platforms*
14	Project File	Fix useEffect dependency array in chat-context.tsx — remove fileSystem to prevent Windows system freeze	*Critical — causes complete OS lockup requiring hard reboot*
15	Course Content	Clarify that # memory shortcut is a convention not a guaranteed system command	*Medium — course teaches inaccurate feature behaviour*
16	Course Content	Add Windows-specific note that Ctrl+V image paste is unsupported — provide file-based workarounds	*Medium — core course workflow completely unavailable on Windows*
17	Project File	Update entire uigen codebase for AI SDK v6 — 5 breaking changes make app non-functional on all platforms	*Critical — app never generates components as published*

Why Windows Compatibility Matters for Anthropic

Anthropic's target sectors for Claude adoption — manufacturing, fintech, and medical services — are enterprise environments. Enterprise environments are overwhelmingly Windows-based. A developer course that does not work on Windows sends a negative signal to exactly the audience Anthropic needs to win over.

As an Anthropic partner in these sectors, Owlstorm.ai will be introducing Claude and Claude Code to clients whose developers work on Windows daily. Having battle-tested Windows setup documentation is not optional for enterprise adoption — it is a prerequisite.

The good news is that all seventeen issues identified in this report are straightforward to address. The most critical finding is Issue 17 — the AI SDK v6 breaking changes make the app completely non-functional on all platforms, not just Windows. This single issue means the course project does not work for any user anywhere as published. Combined with the Windows-specific issues documented in Issues 1 through 16, the course materials require a comprehensive update before they can reliably deliver the learning experience Anthropic intends. We would be happy to assist Anthropic in validating a corrected version of the course materials on Windows 11 as part of our partner onboarding.

About Owlstorm.ai

Owlstorm.ai is a boutique AI consultancy specialising in responsible AI deployment in manufacturing, fintech, and medical services. We are currently completing the Anthropic partner

application process. This report was prepared as part of our technical onboarding and reflects our commitment to contributing practical, real-world feedback to the Anthropic partner ecosystem.

Owlstorm.ai | www.owlstorm.ai | Prepared March 17, 2026